

Domain-specific QA using RAG

Aswath Mandakath Gopinath

Linköping University / Linköping, Sweden

aswma317@student.liu.se

Abstract

In recent period, there has been significant advancement in the area of Question Answering(QA) systems, driven by the integration with powerful machine learning models and large-scale datasets. This paper explores the approach of Retrieval-Augmented Generative(RAG) models for the QA task by leveraging the power of both generative language models and information retrieval techniques.

The paper begins by providing an overview of how a generative model performs on a QA task and its limitation, emphasizing the need for additional knowledge sources and improved methodologies to extract context from these sources to effectively answer complex queries.

Our research delves into the intricacies of RAG and explores its impact on domain-specific QA tasks. Furthermore, we conduct comprehensive experiments in evaluating the RAG model using custom designed dataset and evaluation metrics that evaluates all components of the RAG model. The result demonstrates the effectiveness of RAG in achieving higher accuracy and context-aware responses.

1 Introduction

In recent years, Large-Language Models(LLM) have garnered a lot of attention in the field of machine learning, primarily attributed to the transformer architecture (Vaswani et al., 2023; Naveed et al., 2023), availability of vast and diverse open-source datasets(Williams et al., 2018; Socher et al., 2013; Zhong et al., 2017) and better computational strategies(Hu et al., 2021; Dettmers et al., 2023, 2022). Furthermore, pre-trained LLMs have been shown to learn substantial knowledge from data (Petroni et al., 2019). Consequently, LLMs have become instrumental in Question Answering (QA) tasks, demonstrating prowess in generating quality answers for open-domain questions related to their training data(Tan et al., 2023). However, their performance falters notably when faced with domain

specific questions(Arefeen et al., 2023; Ling et al., 2023), i.e. questions related to data that was not observed during the training of the LLM, leading to instances of hallucination (Marcus, 2020).

Adapting LLMs to downstream domain specific data requires large amount of domain-specific and task-specific data, and acquiring, cleaning and pre-processing this data can be both time and resource intensive(Ling et al., 2023). To address these challenges, this paper delves into the promising paradigm of Retrieval-Augmented Generative models(RAG)(Lewis et al., 2021), which uses a pre-trained retriever and a pre-trained seq2seq(Sutskever et al., 2014) generator model to answer domain-specific questions. This amalgamation enables the QA system to not only access the parametric memory from the pre-trained seq2seq model but also from the non-parametric memory which is usually a vector store.

In the subsequent sections, the paper first articulates the problem statement, and then provides an in-depth exploration of the RAG model's implementation¹ for the identified challenge. We will also shed light on the handicap of not using RAG and show how implementing RAG for this problem provides better results. Employing the Retrieval Augmented Generation Assessment (RAGAs) framework - an innovative tool for reference-free evaluation of RAG pipelines (Es et al., 2023) - we conduct a comprehensive evaluation of our implemented RAG pipeline on a custom-designed dataset.

Through rigorous experimentation and analysis, we seek to underscore the efficacy of RAG in domain-specific QA tasks. Our investigation extends to scenarios where domain-specific data is scarce, rendering traditional fine-tuning approaches impractical. By uncovering the significance of

¹The notebook containing the implementation can be found in github: <https://github.com/ash-rulz/TextMining/blob/main/TextMiningProject.ipynb>

RAG systems in addressing domain-specific questions, we aim to contribute valuable insights to the evolving landscape of QA systems and their adaptability to resource-constrained environments.

2 Problem Statement

Harry Potter and the Philosopher’s Stone (Rowling, 1997) is the first book in the Harry Potter(HP) series. The challenge at hand is to develop a tailored QA system specifically designed for the content within this literary work, which we will consider as the domain-specific data. In comparison to the vast corpus of data available to train LLMs, the intricacies of this work pose a unique hurdle - its data scarcity is akin to the challenges encountered in domain-specific contexts.

The primary issue is that there is a scarcity of well established, open-source, domain/task-specific data related to this literary work. This scarcity hampers conventional fine-tuning approaches for LLMs, as the lack of sufficient training data prevents effective model learning. We could curate a small set of domain/task-specific dataset, however fine-tuning a LLM on this dataset can lead to catastrophic forgetting, where the model forgets its pre-training knowledge and overfits to the domain specific data(Chen et al., 2020).

In the subsequent section of this paper, we delve into the detailed exploration of the implementation and the evaluation of the RAG model as a prospective solution to this problem, aiming to shed some light on its effectiveness in handling domain-specific QA tasks with limited data.

3 Method

The whole idea is encapsulated in Figure 1.

3.1 Vector Database(DB)

The vector store will hold the context to be fetched by the retriever, and since the context length of LLMs are limited, the first step is to perform document chunking(Chase, 2022). The generator (explained in section 3.3) we use in our exercise has a maximum context length of 512 tokens, hence during the chunking of the document, we will restrict each chunk size to 70, as 3 such context will be plugged into the prompt, and the remaining space would be taken up by the remainder of the prompt. The content of the HP book is split into such smaller chunks, and these chunks are converted into contextualized sentence embeddings

using sentence transformer models which are inspired by the Siamese BERT-Networks(Reimers and Gurevych, 2019). The sentence transformer used in this exercise is the all-MiniLM-L6-v2(Face, 2022a) published by Hugging Face(Wolf et al., 2020), which converts the sentence chunks to a dense vector of size 384.

The contextualized embeddings are then stored in a vector store, and here we use a vector store called Faiss(Face, 2022b) provided by Lang Chain(Chase, 2022). It is a library for efficient similarity search and clustering of dense vectors, and was inspired by the Facebook AI Similarity Search(FAISS)(Johnson et al., 2017) paper.

3.2 Retriever

The retriever employs Dense Passage Retrieval(Karpukhin et al., 2020), wherein a question raised by the user is also converted to contextualized embedding which is then used to do a similarity search with the vector store. The top k embeddings are retrieved and are converted back to the original sentences. These chunks of sentences are the context which are fed to the generator to answer the question.

3.3 Generator

T5(Raffel et al., 2020) is an encoder-decoder text-to-text model pre-trained on a mixture of multiple tasks, including QA tasks. For our exercises, we will use the successor of the T5 model - FLAN-T5(Chung et al., 2022) model, which is a T5 model instruction-fine-tuned on a multitude of tasks which includes closed-book QA, because of which this is the best open-source model suited for our problem. We will be using the 783 million parameter flan-t5-large model(Face, 2023) which is available in Hugging Face. The generator is fed with the question and the context retrieved by the retriever, and this information is used by the generator to answer the question posed by the user.

3.4 Evaluation Data

For the evaluation of the whole RAG pipeline, the RAGAs framework is used, and for this, golden standard QA pair was generated² from Hobby-Lark(Group, 2019) website which contained a trivia of questions from the HP book. As it was

²The implementation of how the golden standard QA pairs for the evaluation was generated is available at: <https://github.com/ash-rulz/TextMining/blob/main/EvalDataGenerator.ipynb>

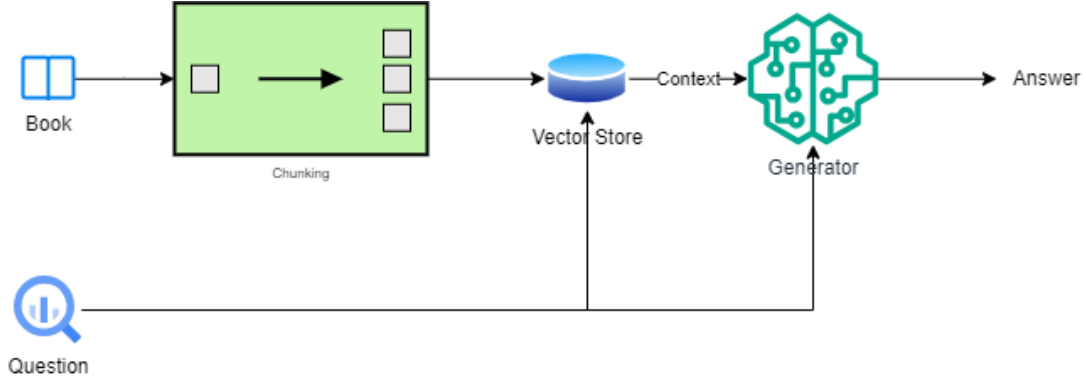


Figure 1: The book is split into smaller chunks, which is then converted to sentence embedding. The sentence embeddings are stored in a vector store. When the user poses a question to the Generator, the question is converted to its embedding representation which is then compared with the vector store for similarity. The closest k embeddings are converted back to text and passed as context to the Generator model for the final answer generation.

difficult to scrape the data from the website directly, the webpage was first converted to a PDF format. The question and answer was extracted from the PDF, and it was used to create a comma separated file. The file contained 141 QA pairs and has two columns - question and ground_truth. A sample of data from the file is shown in the Table 1. Some amount of cleaning has to be done while generating the data, and this was automated, however, some of the data has to be manually corrected, for e.g. unnecessary blank spaces had to be removed, and some of the duplicate letters has to be removed as well. In addition to this, some of the question answer pairs were quite ambiguous and these had to be removed from the evaluation dataset.

question	ground_truth
What is Dumbledore’s first name?	Albus
What does Hagrid give Harry for his eleventh birthday?	A cake
Who was the first person to tell Harry he was a wizard?	Hagrid
What is Ron’s last name?	Weasley
Which house is Draco Malfoy in?	Slytherin
What is the name of the Hogwarts Herbology teacher?	Professor Sprout
What position does Harry Potter play on Gryffindor’s Quidditch team?	Seeker

Table 1: Golden standard evaluation dataset

4 Experiment

We make use of the models and functionalities found under the Transformers library(Wolf et al., 2020) provided by Hugging Face.

The experiment begins by assessing the generator’s performance in zero-shot setting, where no domain-specific context is provided for the QA task. Subsequently, we analyze how the outcomes evolve with the inclusion of supplementary context provided by the retriever. To facilitate this analysis, we segment the HP book into smaller chunks, each containing 70 tokens with a 10-token overlap, resulting in approximately 9197 documents. This chunking process employs the RecursiveCharacterTextSplitter API from LangChain(Chase, 2022). The next step involves transforming these documents into contextualized embeddings using the all-MiniLM-L6-v2 sentence transformer available in Hugging Face. These embeddings are then loaded into a vector store using the FAISS library provided by LangChain(Chase, 2022).

For retriever creation, we utilize the RetrievalQA library from LangChain(Chase, 2022), which retrieves the top 3 documents from the vector store based on the given question. Subsequently, we instantiate the generator by loading the google/flan-t5-large model from Hugging Face. We assess the flan-t5-large model’s ability to answer the query in a zero-shot setting. Additionally, we construct a prompt template using the ChatPromptTemplate library in LangChain(Chase, 2022). This template takes both the context and the question as input.

To orchestrate these components seamlessly, we employ the LangChain expression language(Chase, 2022). The question serves as input to this chain,

traversing through the retriever. The retriever fetches the top 3 documents, and these documents, along with the original question, are integrated into the prompt template as the context and question, respectively. Ultimately, the prompt is fed into the generator, utilizing the provided context to formulate an answer to the question in the prompt.

In this section of the study, we assess the entire RAG pipeline, utilizing the metrics outlined in the RAGAs documentation through the application of the ragas library (Ragas, 2023a). This framework conducts a comprehensive evaluation of the RAG pipeline components, including the retriever and generator, as well as an end-to-end system assessment. A specialized golden source dataset was constructed for this purpose, with specifics provided in the Section [Evaluation Data](#). The ragas library expects data to be in the following structure:

1. Question: the question column from the custom dataset is fed to this column.
2. Contexts: The top 3 documents fetched by the retriever for the particular question is fed into this column.
3. Ground truths: the ground truth column from the custom dataset is fed to this column.
4. Answer: the answer fetched by the generator in the RAG model is fed to this column.

Furthermore, as the library uses OpenAI APIs for some of their internal processing, the OpenAI key must be created and provided. The whole RAG pipeline can then be evaluated on this new dataset using the ragas library.

In the initial phase of our evaluation, the base model—specifically the FLAN-T5—is tested with a retriever component. Importantly, for this phase, the vector database can be regarded as essentially empty, as it is populated with dummy data that is irrelevant to the downstream task. This setup serves as a control condition to simulate a baseline scenario, enabling us to assess the intrinsic capabilities of the base model without the influence of external data. This approach provides a clear understanding of the model's fundamental performance, independent of enhancements from task-relevant data inputs.

Following this, the full RAG model is evaluated, this time using a vector database filled with relevant data for the downstream task. This step is critical

for comparing the performance metrics between the base model and the RAG model, highlighting the added value of integrating relevant data into the retrieval process.

In a further exploration to enhance retrieval scores, an alternative retrieval method termed "parent document retriever" is implemented. In this approach, the document chunk undergoes additional segmentation into smaller components. The rationale underlying this method is the potential for the embeddings of the smaller chunks to more accurately capture the meaning of the statement. During the retrieval process, a similarity search is initially conducted based on these smaller chunks. Subsequently, the corresponding larger parent chunks are fed into the context for the generator. This approach is implemented in an endeavor to optimize the efficiency of the retrieval process and subsequently enhance overall performance scores.

5 Results

As an initial evaluation, the generator is assessed in a zero-shot setting for a question from the domain-specific data. In this scenario, the generator solely utilizes the parametric memory, and the outcome is depicted in Figure 2.

```
1 #Evidence of hallucination by the T5 model
2 question = "What is the name of Filch's cat?"
3 llm_result = llm.invoke(question)
4 llm_result
```

```
'sam'
```

Figure 2: Generator outputs incorrect result for domain-specific question in zero-shot setting. The correct answer for this question is "Mrs.Norris", and the generator is clearly hallucination

In contrast, when employing the RAG model for the question, the generator accesses both the non-parametric memory and the parametric memory. The resulting output is presented in Figure 3.

```
1 #Evidence of how the RAG improved the result
2 result = retrieval_augmented_qa_chain.invoke({"question" : question})
3 result['response']
```

```
'Mrs. Norris'
```

Figure 3: Generator is able to access the non-parametric memory, i.e. the vector store to retrieve the top 3 documents, and is able to get the correct answer.

Finally, the entire RAG pipeline undergoes evaluation utilizing the RAGAs framework. The results

for both the base model and the RAG model are presented in Table 2, showcasing each metric (Ragas, 2023b) alongside its corresponding value.

Component	Metrics	Base Model	RAG Model
Retriever	context_precision	0.00	0.30
Generator	faithfulness	0.00	0.43
Generator	answer_relevancy	0.00	0.63
Retriever	context_recall	0.00	0.64
Retriever	context_relevancy	0.00	0.10
End-to-End	answer_correctness	0.04	0.48
End-to-End	answer_similarity	0.17	0.87

Table 2: RAGAs evaluation results on the base and RAG model.

The comparative analysis indicates that the base model underperforms against the RAG model across all evaluated metrics. Notably, however, the base model registers non-zero values for the metrics 'answer_correctness' and 'answer_similarity'. The RAGAs framework assesses 'answer_correctness' by employing a weighted combination of the semantic and the factual similarity between the generated answer and the ground truth. It is hypothesized that the base model's generation of semantically analogous responses to the ground truth contributed to its non-zero scores. Similarly, 'answer_similarity' quantifies the semantic correspondence between the answer and the ground truth, and for similar reasons mentioned earlier, this metric also recorded a non-zero value.

Although the RAG model is better than the base model on metrics, the model evaluates poorly on 2 metrics - Context Precision and Context Relevancy.

Context relevancy measures the relevancy of the retrieved context, and the low value indicates that the retrieved context does not contain the essential information required to address the provided query.

Context precision measures whether the ground-truth relevant chunks appear as the top ranked documents, and are part of the context.

However, the metrics with respect to the generator and the the end-to-end pipeline is relatively better. Amongst these metrics only faithfulness and answer correctness have the lowest scores. The generator metrics - faithfulness measures the factual consistency of the generated answer against the given context, while the end-to-end metrics - answer correctness measures the accuracy of the generated answer when compared to the ground truth.

Finally, an attempt is made to enhance retriever scores by implementing the parent document re-

triever. Table 3 compares the original RAG approach (Method 1) to the parent document retriever approach (Method 2). Contrary to expectations, there is a considerable decrease in scores across all aspects, indicating that the parent document retriever does not improve results. The retriever's context recall has notably decreased, suggesting that the new retriever fails to retrieve context aligned with the ground truth. This could be attributed to the ineffectiveness of further splitting the document into parent documents in aiding the retrieval process. Consequently, the generator scores and end-to-end scores also experience a decline in performance as a chain reaction.

Component	Metrics	Method 1	Method 2
Retriever	context_precision	0.30	0.22
Generator	faithfulness	0.43	0.27
Generator	answer_relevancy	0.63	0.42
Retriever	context_recall	0.64	0.51
Retriever	context_relevancy	0.10	0.10
End-to-End	answer_correctness	0.48	0.35
End-to-End	answer_similarity	0.87	0.83

Table 3: RAGAs evaluation result comparison between 2 approaches

6 Discussion

In this study, we applied the RAG framework to QA tasks involving domain-specific data. As discussed in Section 5, we illustrated how Language Models (LLMs) tend to generate inaccurate responses when relying solely on their parametric memory. Conversely, when employing the RAG model and providing the LLM access to both parametric and non-parametric memory, correct answers were generated.

Furthermore, we conducted a thorough end-to-end evaluation of the models, encompassing assessments of the retriever, the generator, and the entire model. Our findings reveal that the RAG model outperforms the base model across all metrics, demonstrating significant improvements in performance when non-parametric memory is utilized in addition to parametric memory.

However, despite the satisfactory performance of the generator and the overall evaluation scores of the RAG model, certain retriever metrics such as context relevancy and context precision exhibited low scores. This suggests room for improvement in the retriever component of the RAG model.

Although we attempted to enhance retriever results by implementing an alternative strategy,

namely the parent document retriever, our findings indicate an unexpected adverse impact on the results. This suggests the need for further refinement and exploration of alternative approaches to improve retriever performance within the RAG framework.

As a next step, the exploration of additional options to improve retriever results could be considered. However, it is important to acknowledge that such endeavors would extend beyond the predefined scope of this project. The limitations encountered in improving retriever performance through the parent document retriever approach prompt a cautious approach to future modifications. Further investigations and potential refinements in retriever strategies could be pursued in subsequent research endeavors.

7 Conclusion

In this paper, we have shown how in scenarios where domain-specific data is less, we could employ RAG to get more factual and grounded answers. In such scenarios, when less data prevents us from fine-tuning or pre-training the LLMs, RAG becomes a prospective solution without the additional resource demand and long training hours. The LLM is able to use both the parametric memory and the non-parametric memory to give faithful answers, and the model will get better results as more non-parametric memory is available at its disposal. This is evident from the substantial performance gain observed when comparing the baselined model with the RAG model.

Furthermore, we have illustrated how the RAGs framework can be easily adapted to evaluate all components of the RAG model, which is crucial for comprehensively explaining the overall model performance. This comprehensive evaluation approach enhances our understanding of the model's capabilities and limitations in real-world applications.

8 Future Work

In section 5, it is observed that the relatively poor performance of the retriever limits the overall performance of the RAG model. As further extension to this work, we could employ some of the below strategies to better the performance of the retriever:

1. We converted chunks of data to contextualized embeddings using sentence transformers

to build the vector store, but we could also explore the possibility of using knowledge graphs to better capture the inherent relationship in the data in the form of entities and relationship. Various properties of the nodes in the graph are transformed into embeddings and added to the vector stores, and using these vector stores, we could get the k closest nodes for the query raised by the user. This strategy could ensure better context fetched by the retriever, and could improve the overall performance of the model.

2. We could also create metadata from the data used to load the non-parametric memory, and during the similarity search, the metadata could be searched first using filters, and then a similarity search could be done between the contextualized embeddings and the query embedding. This would ensure better context fetched by the retriever.

With respect the evaluation, although RAGAs framework provides a whole package to evaluate the RAG model, the shortcoming of this approach is that we have to pay for the use of the OpenAI API by the framework. We could explore the possibility of using open-source frameworks as an alternative for this approach.

References

- Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. 2023. [Leancontext: Cost-efficient domain-specific question answering using llms](#).
- Harrison Chase. 2022. [LangChain](#).
- Sanyuan Chen, Yutai Hou, Yiming Cui, Wanxiang Che, Ting Liu, and Xiangzhan Yu. 2020. [Recall and learn: Fine-tuning deep pretrained language models with less forgetting](#).
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022. [Scaling instruction-finetuned language models](#).
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. [Llm.int8\(\): 8-bit matrix multiplication for transformers at scale](#).

- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. [Qlora: Efficient finetuning of quantized llms](#).
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. [Ragas: Automated evaluation of retrieval augmented generation](#).
- Hugging Face. 2022a. all-minilm-l6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> [Accessed: (2024)].
- Hugging Face. 2022b. Faiss. <https://python.langchain.com/docs/integrations/vectorstores/faiss> [Accessed: (2024)].
- Hugging Face. 2023. flan-t5-large. <https://huggingface.co/google/flan-t5-large> [Accessed: (2024)].
- The Arena Group. 2019. Hobby lark. <https://hobbylark.com/party-games/180-Printable-Trivia-Questions-for-Harry-Potter-and-the-Sorcerers-Stone> [Accessed: (2024)].
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. [Lora: Low-rank adaptation of large language models](#).
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. [Billion-scale similarity search with gpus](#).
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. [Dense passage retrieval for open-domain question answering](#).
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#).
- Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, Junxiang Wang, Tanmoy Chowdhury, Yun Li, Hejie Cui, Xuchao Zhang, Tianjiao Zhao, Amit Panalkar, Wei Cheng, Haoyu Wang, Yanchi Liu, Zhengzhang Chen, Haifeng Chen, Chris White, Quanquan Gu, Jian Pei, and Liang Zhao. 2023. [Domain specialization as the key to make large language models disruptive: A comprehensive survey](#).
- Gary Marcus. 2020. [The next decade in ai: Four steps towards robust artificial intelligence](#).
- Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2023. [A comprehensive overview of large language models](#).
- Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. [Language models as knowledge bases?](#) In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China. Association for Computational Linguistics.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. [Exploring the limits of transfer learning with a unified text-to-text transformer](#).
- Ragas. 2023a. Ragas. <https://docs.ragas.io/en/latest/index.html> [Accessed: (2024)].
- Ragas. 2023b. Ragas metrics. <https://docs.ragas.io/en/latest/concepts/metrics/index.html> [Accessed: (2024)].
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-bert: Sentence embeddings using siamese bert-networks](#).
- J. K. Rowling. 1997. *Harry Potter and the Philosopher's Stone*, 1 edition, volume 1. Bloomsbury Publishing, London.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. [Recursive deep models for semantic compositionality over a sentiment treebank](#). In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, Seattle, Washington, USA. Association for Computational Linguistics.
- Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. [Sequence to sequence learning with neural networks](#).
- Yiming Tan, Dehai Min, Yu Li, Wenbo Li, Nan Hu, Yongrui Chen, and Guilin Qi. 2023. [Can chatgpt replace traditional kbqa models? an in-depth analysis of the question answering performance of the gpt llm family](#).
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. [Attention is all you need](#).
- Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. [A broad-coverage challenge corpus for sentence understanding through inference](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122, New Orleans, Louisiana. Association for Computational Linguistics.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu,

Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. [Huggingface's transformers: State-of-the-art natural language processing](#).

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. [Seq2sql: Generating structured queries from natural language using reinforcement learning](#).